Memoria del proyecto · Actividad 2

Proyecto: Taskflow — gestión visual de tareas

Tecnología: Ionic, Angular, TypeScript y Supabase

Fecha de corte: 28 de octubre de 2026

Estado: Autenticación, operaciones de tareas, actividad, gráficos y tema oscuro implementados; compilación, lint y pruebas automatizadas correctos.

1. Resumen ejecutivo

Desde la memoria de la Actividad 1 se completaron los flujos principales de gestión de tareas y se mejoró el panel. La aplicación permite iniciar sesión y registrarse con Supabase Auth; consultar tareas y actividad; crear, editar y eliminar tareas; cambiar su estado; buscar y filtrar; y recibir actualizaciones mediante Supabase Realtime.

También se actualizaron las visualizaciones para reflejar los datos cargados, se corrigió el espacio de desplazamiento de la lista, y se añadió un tema oscuro persistido en el navegador. Se reparó el TaskService para que use el servicio Supabase que realmente existe en la aplicación y para que sus consultas coincidan con el esquema compartido.

Validación local de este corte: compilación Angular correcta, lint correcto y **32 pruebas automatizadas aprobadas**. Esto valida el código local; no equivale a una prueba de despliegue ni a una comprobación independiente de las políticas de la instancia remota de Supabase.

2. Arquitectura y conexión con Supabase

- SupabaseClientService descarga assets/supabase-config.json, valida que contenga una URL HTTPS y una clave pública, y crea una única instancia reutilizable de Supabase.
- La configuración con credenciales locales no se incluye en el repositorio. Existe una plantilla de ejemplo; cada entorno debe publicar su propio supabase-config.json.
- AuthService implementa lectura y observación de sesión, registro, inicio y cierre de sesión. El registro envía full_name en los metadatos de Auth.
- La ruta privada /home utiliza un guard de autenticación.
- TaskDataService es el servicio que usa el panel y concentra sus operaciones de tareas, historial y canal Realtime.
- TaskService es un servicio separado con sus propias señales y operaciones; no sustituye al TaskDataService que actualmente usa el panel.

Los errores de configuración, autenticación, permisos y consultas se propagan para que la interfaz pueda mostrarlos. No se guarda una clave service_role en el cliente.

3. Esquema de datos y auditoría

La aplicación se ajusta a las tablas public.tasks y public.activity_logs que compartió el usuario:

tasks

Incluye identificador UUID, título, descripción, estado, prioridad, datos de asignación, creador, subtareas y etiquetas JSONB, horas estimadas y registradas, y marcas de tiempo. El esquema restringe estados a TODO, IN_PROGRESS y COMPLETED; prioridades a LOW, MEDIUM y HIGH; arrays JSON a tipo array; y horas a valores no negativos.

Los índices existentes o incluidos en la migración cubren usuario asignado, creador, estado y fecha de creación. Un trigger mantiene updated_at.

activity_logs

Almacena acción, usuario, correo, detalles JSON y fecha de creación, con un task_id opcional. El trigger log_task_activity() registra creación, actualización y eliminación de tareas. Las actualizaciones indican los campos modificados y excluyen updated_at.

La migración de auditoría revoca la inserción directa de logs a los roles cliente y hace que el trigger sea quien genere los eventos. Las políticas RLS de las migraciones limitan la lectura y modificación de tareas a sus creadores o usuarios asignados, permiten borrar al creador y restringen la lectura de actividad.

El usuario indicó que ejecutó los scripts SQL en Supabase. No se inspeccionó directamente la instancia remota durante esta validación, así que las políticas y los triggers remotos deben considerarse confirmados por el usuario, no verificados desde este entorno.

4. Funcionalidades de la aplicación

Autenticación

- Inicio de sesión y registro con correo y contraseña.
- Sesión mantenida por Supabase Auth y perfil de usuario visible en el panel.
- Cierre de sesión y protección de la ruta privada.
- Errores de autenticación y configuración presentados como mensajes de aplicación.

Gestión de tareas

- Carga ordenada por fecha de creación.
- Creación desde el formulario del panel.
- Edición de título, descripción, estado, prioridad, etiquetas y horas.
- Eliminación de tareas creadas por el usuario, respetando las políticas RLS.
- Cambio rápido entre completada y pendiente.
- Búsqueda por título, descripción y etiquetas; filtros por estado.
- Métricas derivadas de las tareas cargadas: totales, estados y horas.

Actividad y sincronización

- Consulta del historial reciente de activity_logs.
- Suscripción Realtime a cambios de tasks, con actualización de tareas e historial.
- Gestión de errores de canal y liberación de la suscripción al destruir la página.

Gráficos, desplazamiento y apariencia

- Gráfico semanal SVG calculado a partir de created_at; cuenta las tareas creadas en los siete días de la semana mostrada.
- Gráfico de dona por estado con los valores de las tareas cargadas. El SVG declara explícitamente el relleno transparente y aplica la rotación necesaria para que el anillo se represente correctamente.
- Desplazamiento vertical dentro de la lista de tareas, en lugar de aplicarlo al título; los filtros de estado no requieren desplazamiento horizontal.
- Modo oscuro y claro, con preferencia guardada en localStorage bajo taskflow-theme.
- Estilos basados en variables de tema para mantener coherentes los colores de la interfaz.

5. Reparación del TaskService

Se corrigieron los defectos principales detectados:

1. Se reemplazó la dependencia de un SupabaseService inexistente por SupabaseClientService.getClient(), usado en el resto de la aplicación.

2. Se eliminó la consulta por due_date de getTasksByMonth(), porque esa columna no pertenece al esquema suministrado. El método filtra por límites UTC de created_at y rechaza meses inválidos.

3. Se añadió TaskMapper para convertir filas de base de datos a Task y preparar escrituras con nombres de columna compatibles.

4. El mapeo normaliza JSONB de etiquetas y subtareas, estados, prioridades y horas. Cuando una subtarea persistida no tiene identificador, crea uno derivado de su posición.

5. Se aseguró que las tareas tengan siempre el array de subtareas requerido por el modelo de dominio.

6. Las operaciones de actualización y borrado detectan cuando Supabase no devuelve una fila, lo que puede deberse a RLS o a que la tarea no existe.

7. El canal Realtime informa errores y ofrece limpieza explícita.

El método getTasksByMonth(year, month) ahora se refiere al mes de **creación**, no a una fecha límite.

6. Límites conocidos y siguientes pasos

- El esquema actual no incluye due_date ni organization_id. Por eso la interfaz muestra «Sin fecha límite» y el mapeador no envía esos campos a Supabase. Para soportar calendario por vencimiento o multi-organización haría falta acordar y aplicar una migración de esquema, junto con sus políticas RLS.
- La visibilidad de tareas depende de RLS. Un usuario solo verá filas que las políticas remotas permitan devolver.
- La suscripción Realtime requiere que tasks esté incluida en la publicación supabase_realtime; la migración intenta añadirla cuando la publicación existe.
- Se recomienda validar en el entorno desplegado el archivo assets/supabase-config.json, las políticas con más de una cuenta y las operaciones CRUD reales.
- Los dos servicios (TaskDataService y TaskService) mantienen responsabilidades cercanas. El panel usa actualmente TaskDataService; si TaskService se convierte en el servicio oficial, conviene consolidar o retirar la duplicación en un cambio separado y probado.

7. Validación de este corte

Comprobación	Resultado
npm test -- --watch=false	Correcto: 5 archivos y 32 pruebas aprobadas
npm run build	Correcto: bundle de producción generado en www
npm run lint	Correcto: todos los archivos pasan
git diff --check	Correcto: sin errores de espacios en los cambios revisados

Las pruebas incluyen autenticación/configuración, carga y mapeo de tareas, operaciones CRUD, filtros, Realtime, tema y gráficos. La validación no ejecuta acciones contra la instancia real de Supabase.

8. Archivos de referencia

- src/app/core/supabase/supabase-client.service.ts: carga de configuración y cliente Supabase.
- src/app/core/auth/auth.service.ts: sesión y autenticación.
- src/app/core/tasks/task-data.service.ts: datos usados por el panel.
- src/app/core/services/task.service.ts: servicio reparado y pruebas específicas.
- src/app/core/mappers/task.mapper.ts: traducción entre filas Supabase y modelo de dominio.
- src/app/home/home.page.ts: estado del panel, métricas, gráficos, operaciones y tema.
- supabase/migrations/20261027170000_create_task_tables.sql: tablas, RLS, índices, trigger de actualización y publicación Realtime.
- supabase/migrations/20261027175000_restrict_activity_log_inserts.sql: trigger de auditoría y restricción de inserción de logs.